Name: . . . . . Number: . . . . . .

CSC211 Data Structures and Algorithms

Uses of Stacks and Queues

Term 1 Tutorial 5 — 12 March 2026

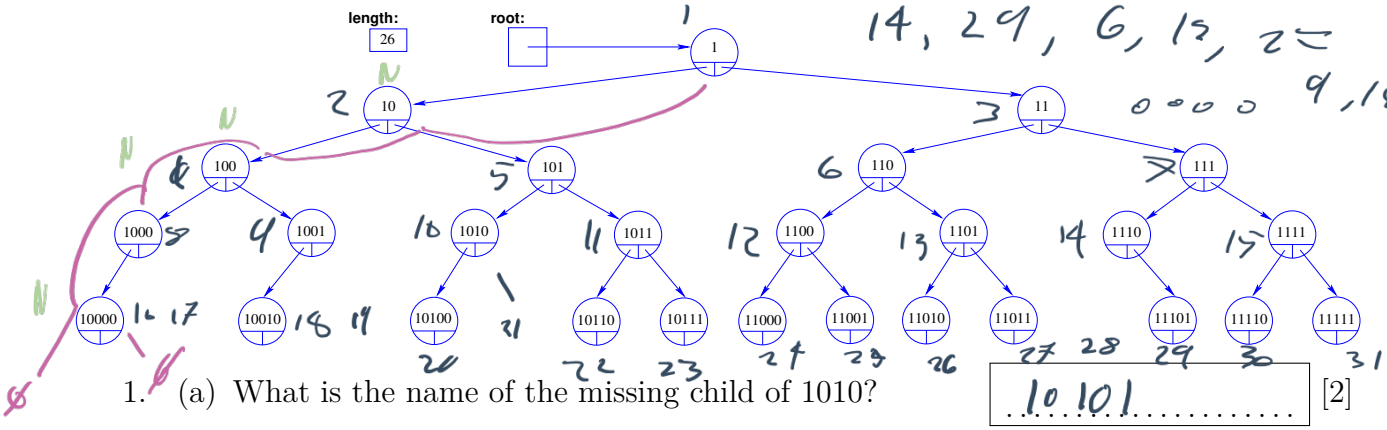
Submit today before leaving tutorial

Consult your tutors, Gemini, DeepSeek, ChatGPT, Claude, or Grok.

Today we experience the use of stacks and queues. Hanoi's disks behave like a *stack*—operations on a stack are done on the topmost node using **pop**, **push** or **peek**. In a *queue* nodes are entered at its rear with **enqueue** and removed from the from the front with **dequeue**, or one can just **peek** at it. Both structures can be tested with the predicates **isEmpty** and **isFull**.

Consider the tree structure **T** given in the figure and answer the questions. The nodes of **T** are named according to a simple scheme that is related to their position in **T**. These names should be used to answer the questions. The root or topmost node is called 1. The number of bits in each name determines the level of the node. Do you understand the meaning of the bits? A node above another is a *parent* of the *child* node below it. *Left* children have their parent's name followed by a zero, and *right* children have the parent's name followed by a one. Note that a parent can have at at most two children.

1, 3, 7, 15, 31, 30, 14, 29, 6, 13, 27, 9, 18, 8, 16



1. (a) What is the name of the missing child of 1010? 10101 [2]

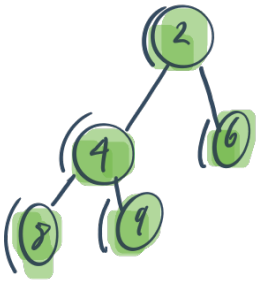
(b) What is the name of the only missing left node? 11100 [2]

(c) A "visit" **N** means that the node is visited and its value is printed. An **L** indicates a left traversal to the left subtree and **R** indicates a traversal to the right subtree. The **LNR**—in-order—traversal of the tree **T**, writing the names in decimal, is: $16, 8, 4, 18, 9, 2, \ldots$ What are the last four nodes that are visited?

20 [2] 16 [2] 5 [2] 22 [2].

(d) Write the last four nodes of the **NRL** traversal of the tree.

9 [2] 18 [2] 8 [2] 16 [2].



"LNR"

2 $\xrightarrow{L}$ 4 $\xrightarrow{L}$ 8 $\xrightarrow{L}$ ∅

print(8) $\xrightarrow{R}$ ∅

print(4) $\xrightarrow{R}$ 9 $\xrightarrow{L}$ ∅

print(9) $\xrightarrow{R}$ ∅

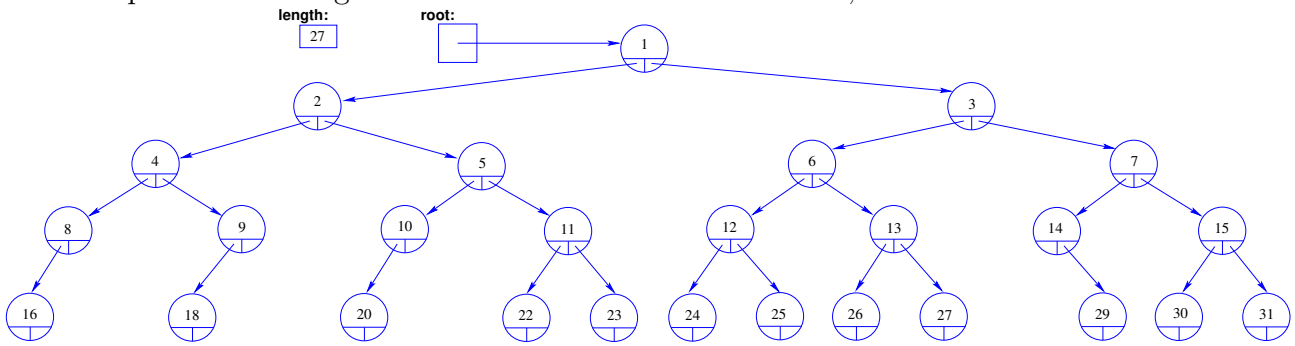
print(2) $\xrightarrow{R}$ 6 → ∅ → ∅

print(6)

"Trees are recursive structures"

(e) The tree **T** can be traversed in 6 possible ways. [2]

2. **Another approach to traversal**
Another way of considering the traversal of trees such as **T** is by employing a stack or a queue for storing the nodes that must still be treated,



DepthFirstTreeTraversal *push* the root node onto stack. **do** *visit* and *remove* the top of the stack: *visit* by pushing top's children onto the stack. **repeat** until the stack is empty.

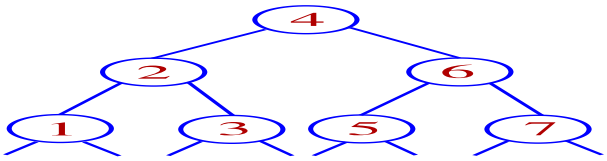
The **depth-first** traversal of the small tree below is: $4, 2, 1, 3, 6, 5, 7$.
Give a list of the last eight nodes visited in a depth-first traversal of the tree on the top of the page.

1, 2, 4, 8, 16, 9, 18, 5 [8].

3. Tree traversal—**breadth-first** Using a **queue** instead of a *stack* turns the above algorithm into **breadth-first traversal**.

BreadthFirstTreeTraversal *add* the root node into the queue. **do** *visit* and *remove* the front of the queue: *visit* by adding front's children into the queue. **repeat** until the queue is empty.

The **breadth-first** traversal of this tree



is: $4, 2, 6, 1, 3, 5, 7$.
Now give a list of the first eight nodes visited in a breadth-first traversal of the tree on the top of the page.

1, 2, 3, 4, 5, 6, 7, 8 [8].